Operating Systems Orange Problem VCS

Ojas Taori – PES1UG24CS308

F section

Report

Screenshots:

1A)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

2A)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$  ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ find .pes/objects -type f
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/cd8895fe1db1950ae1cc9493530772b986759adaa151dcf7c3953ad5a6c7ab
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ xxd .pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39 | head -20
00000000: 626c 6f62 2036 0048 656c 6c6f 0a         blob 6.Hello.
```

3A)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes add file1.txt file2.txt
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  tree.c
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  index.h
  untracked:  commit.c
  untracked:  README.md
  untracked:  index.c
  untracked:  Makefile
  untracked:  tree.h
  untracked:  test_objects
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  test_tree
```

3B)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776521325 6
 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776521325 6
 file2.txt
```

4A)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ echo "Hello" > hello.txt
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes add hello.txt
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 74cd8895fe1d... Initial commit
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ ./pes log
commit 74cd8895fe1db1950ae1cc9493530772b986759adaa151dcf7c3953ad5a6c7ab
Author: Ojas Taori PES1UG24CS308
Date:   1776522087

    Initial commit
```

4B)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/74/cd8895fe1db1950ae1cc9493530772b986759adaa151dcf7c3953ad5a6c7ab
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/refs/heads/main
```

4C)

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ cat .pes/refs/heads/main
74cd8895fe1db1950ae1cc9493530772b986759adaa151dcf7c3953ad5a6c7ab
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final

```
pes1ug24cs308@pes1ug24cs308:~/PES1UG24CS308-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: e0edb4a16122... Initial commit

Log after first commit:
commit e0edb4a16122b628bbb9274d32c9f51702a740b920f47983e2054984972593db
Author: Ojas Taori PES1UG24CS308
Date:   1776522141

    Initial commit


--- Second Commit ---
Committed: b94c60b03812... Update file.txt

--- Third Commit ---
Committed: 7eebee815720... Add farewell
```

```
--- Full History ---
commit 7eebee81572020b1c0d5dc0a600c1235e3a699865195f15c549f574090f8f172
Author: Ojas Taori PES1UG24CS308
Date:   1776522141

    Add farewell

commit b94c60b0381246e3c7048fb4bf112d020b1e18dfbb4c3bf461958f3852d1453f
Author: Ojas Taori PES1UG24CS308
Date:   1776522141

    Update file.txt

commit e0edb4a16122b628bbb9274d32c9f51702a740b920f47983e2054984972593db
Author: Ojas Taori PES1UG24CS308
Date:   1776522141

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7eebee81572020b1c0d5dc0a600c1235e3a699865195f15c549f574090f8f172

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7e/ebee81572020b1c0d5dc0a600c1235e3a699865195f15c549f574090f8f172
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b9/4c60b0381246e3c7048fb4bf112d020b1e18dfbb4c3bf461958f3852d1453f
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e0/edb4a16122b628bbb9274d32c9f51702a740b920f47983e2054984972593db

=== All integration tests completed ===
```

5.1)

To implement `pes checkout <branch>`, first read `.pes/refs/heads/<branch>` to resolve the target commit and tree, compare it with the current HEAD snapshot, and run a conflict check before touching files; if no conflicts exist, rewrite the working directory to exactly match the target tree (create, overwrite, delete, and apply modes), rebuild `.pes/index` to reflect that snapshot, and finally update `.pes/HEAD` to `ref: refs/heads/<branch>`. This operation is complex because it must transform one full filesystem state into another without losing local work, handle file-vs-directory path collisions, and ideally behave atomically so partial failures do not leave an inconsistent repository.

5.2)

To detect a dirty work tree checkout conflict using only index and object storage, build path maps for current HEAD tree, target branch tree, and index, then scan indexed paths in the working directory using stat metadata and content hashing when needed to identify locally modified or deleted files; for every locally changed path, if the target branch also changes that same path relative to current HEAD (or changes its type between file and directory), mark it as a conflict and abort checkout with the conflicting paths listed. This ensures checkout is refused only when local uncommitted work would actually be overwritten.

5.3)

In detached HEAD state, `.pes/HEAD` points directly to a commit hash instead of a branch reference, so new commits are still created normally and HEAD advances, but no branch pointer is updated, meaning that history can become unreachable after switching away. Recovery is simple as long as you still know the commit hash: create a new branch reference file (for example `.pes/refs/heads/recover`) containing that hash and then repoint HEAD to that branch so the commits become permanently reachable again.

6.1)

Use a mark-and-sweep garbage collector: start from all branch refs (and detached HEAD if present), traverse reachable commit, tree, and blob objects with DFS or BFS, and store visited hashes in a hash set for fast lookup, then scan `.pes/objects` and delete every object not in that reachable set. The runtime is O(R + O), where R is objects reached from refs and O is total stored objects scanned, and for a repository with 100,000 commits and 50 branches, commit traversal is usually near the count of unique reachable commits (often close to 100,000 because histories share ancestors), while total visited objects becomes commits plus associated trees and blobs.

6.2)

Concurrent GC is dangerous because commit creation writes objects before updating refs, so a GC that marks only from current refs can temporarily classify those new objects as unreachable and delete them, after which the commit may update a ref that points to missing data and corrupt history. Real systems reduce this risk by combining lock-based or atomic ref updates with conservative pruning policies, such as grace periods and additional retention roots (for example reflogs or in-progress references), so freshly written objects are not collected during active writes.